

swtpcsim — Learning the symbolic debugger

2026-09-27 · 0e4d89e

A 44-byte 6800 program, its symbols, and a walk through the monitor's debugger — load it, disassemble it by name, single-step it, break on a label, and run it until it prints.

```
swtpcsim examples/debugger/debugger.toml
```

You land at the `swtpcsim>` prompt with an empty machine: a Motorola 6800, 32K of RAM, and an MP-S serial console. There is no monitor ROM and no `startup`, so nothing runs yet — that is the first thing you will do, and the point of the exercise is that you do it all by hand. Here the `swtpcsim>` monitor is the bus master.

The program is `HELLO.ASM`: it prints `HELLO, WORLD` through the MP-S (a 6850 ACIA) and then `WAI`s — the 6800's "wait for interrupt", which parks the processor and, on a real machine, hands control back to the ROM monitor. It is small on purpose, so that every label, every `EQU`, and every kind of operand is right there to be looked at.

1 — Load the symbols, then the program

Two separate files, because they are two separate things. `HELLO.LST` is the assembler's **listing** — it is where the *names* live. `HELLO.S19` is the assembled **bytes**, in Motorola S-record form. Loading the symbols does not put a single byte in memory; loading the S-record does not teach the monitor a single name.

```
swtpcsim> SYMBOLS LOAD HELLO.LST
12 symbol(s) from HELLO.LST
swtpcsim> LOAD HELLO.S19
loaded 44 bytes (1 page) from HELLO.S19 (0100-012B)
start address 0000 -- LOAD does not set the PC (EXAMINE an address to run there)
```

`LOAD` puts the bytes in memory and stops there. The S-record names an entry point, but putting a program in memory and *running* it are two separate operator acts — you set the PC yourself, below, with `EXAMINE`.

2 — Disassemble it, and read it by name

`DISASM` takes a range, and a symbol is accepted anywhere an address is — so name the range:

```
swtpcsim> DISASM START-DONE
START:
0100 8E 01 3C LDS #STACK
0103 CE 01 1D LDX #MSG
LOOP:
0106 A6 00 LDAA 00,X
0108 27 05 BEQ DONE
010A 8D 04 BSR PUTC
010C 08 INX
010D 20 F7 BRA LOOP
DONE:
010F 3E WAI
```

Three things are happening on top of the plain hex, and each has a rule worth knowing.

A label heads its own line. `START:`, `LOOP:`, `DONE:` — the way an assembler listing prints them, so a branch destination announces itself where it lands.

An operand reads as the name it points at. `BEQ DONE`, `BSR PUTC`, `BRA LOOP` — every branch target is shown as a symbol; and `LDX #MSG` shows the 16-bit address the instruction loads.

`LDS #STACK` is annotated even though `STACK` is an `EQU`, not a label. A program *label* heads a line, but an *operand* is a value the instruction points at — and there an `EQU` that is really an address is exactly what you want to see. (A real label still wins when both share a value.)

Now disassemble the subroutine, and watch what does **not** get named:

```
swtpcsim> DISASM PUTC 7
PUTC:
0110 36 PSHA
PWAIT:
0111 F6 80 04 LDAB ACIAS
0114 C4 02 ANDB #02
0116 27 F9 BEQ PWAIT
0118 32 PULA
0119 B7 80 05 STAA ACIAD
011C 39 RTS
```

`LDAB ACIAS` and `STAA ACIAD` name their operands: those are **extended** (16-bit) addresses, and `ACIAS` / `ACIAD` are `EQU`s whose values (`8004` / `8005`) really are addresses. But `ANDB #02` is **not** shown as

`ANDB #TDRE`, even though `TDRE EQU $02` is loaded (`SHOW SYMBOLS TDRE` proves it). That is deliberate: an **immediate byte** is a *value*, not an address, and only a 16-bit operand is treated as an address. So the bit mask stays `02`, while `BEQ PWAIT` right below it still reads as the label, because *that* operand is an address.

3 — Single-step, and watch the registers

`EXAMINE` loads the program counter — it is the front-panel switch that jams an address into the PC — so `EXAMINE START` puts you at `0100`. Then `STEP` runs one instruction and shows the machine, with the next instruction disassembled symbolically:

```
swtpcsim> EXAMINE START
0100 8E . 10001110
H0I1N0Z0V0C0 A=00 B=00 X=0000 SP=0000 PC=0100 LDS #STACK
swtpcsim> STEP 3
H0I1N0Z0V0C0 A=00 B=00 X=0000 SP=013C PC=0103 LDX #MSG
H0I1N0Z0V0C0 A=00 B=00 X=011D SP=013C PC=0106 LDAA 00,X
H0I1N0Z0V0C0 A=48 B=00 X=011D SP=013C PC=0108 BEQ DONE
```

The flags at the left are the 6800's condition codes — **H I N Z V C** (half-carry, interrupt mask, negative, zero, overflow, carry) — each shown with its bit. Read down the register line: `SP=013C` after `LDS #STACK` set up the stack, `X=011D` (that is `MSG`) after `LDX #MSG`, and `A=48` after `LDAA 00,X` — `48` is `'H'`, the first byte of the string. `STEP` with no count does one instruction; `NEXT` is the same but runs a `BSR / JSR` to its return instead of descending into it.

4 — Break on a name, and run

A breakpoint takes a symbol, too. Set one on the subroutine and run:

```
swtpcsim> EXAMINE START
0100 8E . 10001110
H0I1N0Z0V0C0 A=48 B=00 X=011D SP=013C PC=0100 LDS #STACK
swtpcsim> BREAK PUTC
breakpoint 1: pc      0110
swtpcsim> RUN
breakpoint 1 (pc      0110) -- stopped at 0110
H0I1N0Z0V0C0 A=48 B=00 X=011D SP=013A PC=0110 PSHA
```

It stopped the first time the program reached `PUTC` , with `A=48` — the `'H'` it was about to send. Clear it and set one on `DONE` instead, and this time let it finish:

```
swtpcsim> NOBREAK 1
breakpoint 1 cleared.
swtpcsim> EXAMINE START
0100 8E . 10001110
H0I1N0Z0V0C0 A=48 B=00 X=011D SP=013A PC=0100 LDS #STACK
swtpcsim> BREAK DONE
breakpoint 1: pc      010F
swtpcsim> RUN
HELLO, WORLD

breakpoint 1 (pc      010F) -- stopped at 010F
H0I1N0Z1V0C0 A=00 B=02 X=012B SP=013C PC=010F WAI
```

There it is: the program ran, printed `HELLO, WORLD` through the MP-S, and stopped exactly at `DONE` — the byte *before* it would have executed the `WAI` . Type `RUN` once more with no breakpoint set and it runs the `WAI` :

```
swtpcsim> RUN
WAI -- the processor is parked at 0110 waiting for an interrupt, and no board is pulling IRQ or NMI.
```

That is the machine waiting for you. On a machine with a monitor ROM the `WAI` would hand back to the monitor's `.` or `$` prompt; here there is no ROM and nothing pulling an interrupt line, so it simply parks — which is the 6800's version of a program that has finished and given the console back.

5 — A few more you now have

Command	Try
DUMP MSG	the string, dumped by name — <code>DUMP</code> resolves a symbol but does not annotate the bytes, because there is no instruction there to say which are addresses
SHOW SYMBOLS	all twelve; the <code>=</code> column marks the <code>EQU</code> constants (<code>ACIAS</code> , <code>TDRE</code> , <code>STACK</code>) apart from the code/data labels (<code>START</code> , <code>LOOP</code> , <code>MSG</code>). <code>SHOW SYMBOLS PW*</code> filters with a glob
BREAK PUTC IF A==2C	a conditional breakpoint — stop at <code>PUTC</code> only when it is about to send <code>2C</code> , the <code>,</code> ; the machine prints <code>HELLO</code> and stops there
TRACE ON / HISTORY	log every instruction as it runs, or show the run-up to a stop

DUMP MSG shows the string sitting where LDX #MSG pointed:

```
swtpcsim> DUMP MSG
0110                                48 45 4C                HEL
0120  4C 4F 2C 20 57 4F 52 4C  44 0D 0A 00 8A 5D CB DD  L0, WORLD....]..
```

SYMBOLS CLEAR forgets the names again; disassemble after that (DISASM 0100 — with the names gone you are back to typing the address) and every operand is plain hex once more, which is a good way to see exactly how much the symbols were buying you.

The files

File	What it is
debugger.toml	The machine: a 6800, 32K of RAM, and an MP-S console. No ROM, no disk — nothing you do not need to single-step a program.
HELLO.ASM	The source, for reading. Labels, EQU s, a loop, and a subroutine.
HELLO.LST	The assembler listing — the file SYMBOLS LOAD reads. Labels feed the name→line and name→operand annotation; EQU s feed operands only.
HELLO.S19	The assembled bytes , in Motorola S-record form — the file LOAD reads.

HELLO.ASM and HELLO.LST are the same program seen two ways: the listing is the source with the assembler's address and object-code columns added on the left. Edit the source and reassemble (with a 6800 cross-assembler such as as0), and both the bytes and the names move together.